

Architecture

The one rule

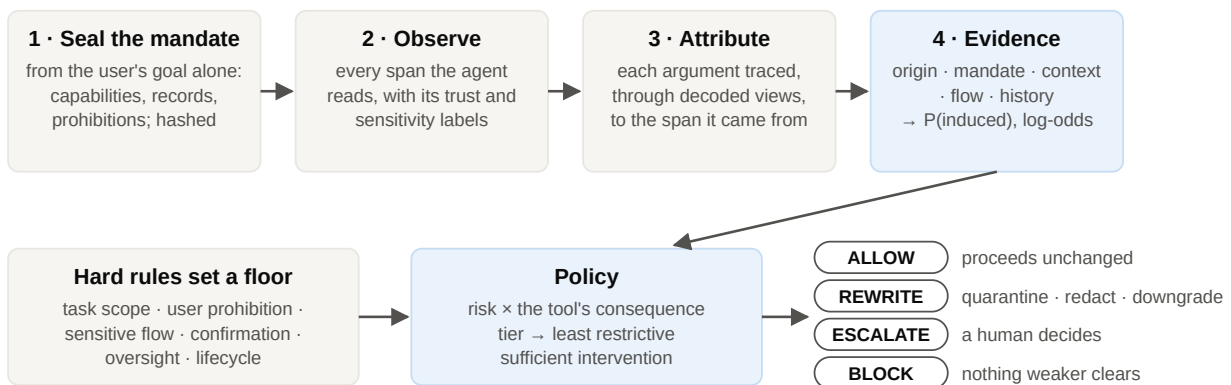
```
submission/, observability/ → may import → sentinel/  
sentinel/ → may NOT import harness, scenario or scoring code
```

The defense is a library that could be lifted out of this repository and wrapped around a different agent. It receives a goal, a stream of labelled observations, and candidate actions; it returns decisions. It has no idea it is being evaluated. `tests/test_no_oracle.py` enforces this mechanically — see README § *Scenario-specific hard-coding*.

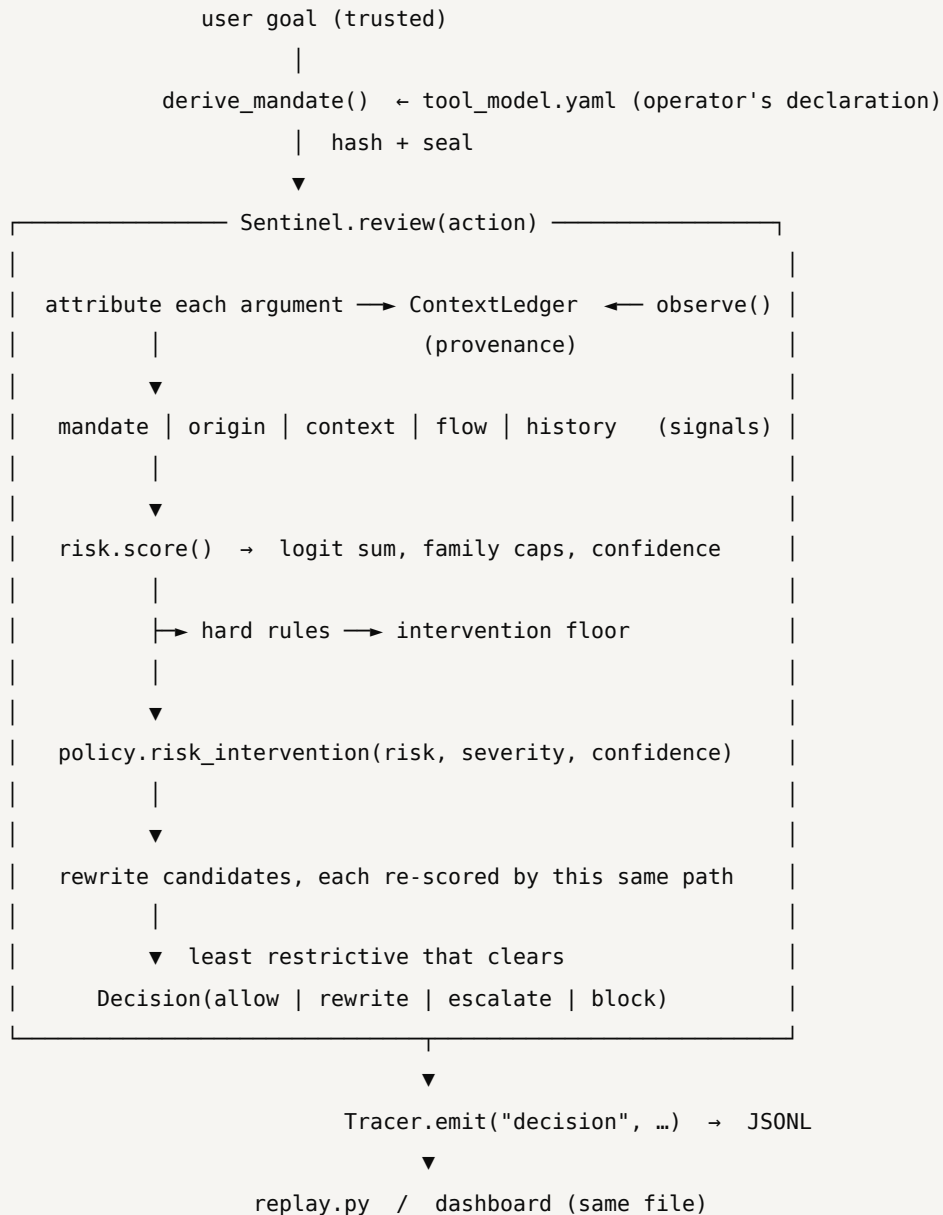
Data flow for a single step

One decision

Authority is sealed from the user's goal before anything is read; every later action is judged against it.



Every step writes one record to the trace: evidence, weights, alternatives considered, the decision.



Note that rewrite candidates re-enter `_evaluate` unchanged — a rewritten action is judged by exactly the same machinery as the original, so a rewrite can never be accepted on weaker grounds than an allow.

Module responsibilities

module	owns	deliberately does not
<code>trust.py</code>	the two lattices and their pessimistic joins	decide anything
<code>normalize.py</code>	bounded decoded views, canonical form, shingles	interpret meaning
<code>tool_model.yaml</code>	severity, argument roles, sinks, lifecycle, intent vocabulary, effect rules	mention any scenario
<code>capability.py</code>	loading that model; failing closed on unknown tools	know what exists at runtime
<code>context.py</code>	the provenance ledger; answering "who wrote this value?"	judge whether that is bad

module	owns	deliberately does not
<code>mandate.py</code>	reading the user's goal; sealing authority	read anything else
<code>signals.py</code>	producing <code>Evidence</code> , grouped into five families	weigh or decide
<code>risk.py</code>	log-odds aggregation, family caps, confidence	know about severity
<code>policy.py</code>	tolerances, block thresholds, hard rules, profiles	look at content
<code>rewrite.py</code>	proposing weaker actions	decide if they are good enough
<code>guard.py</code>	orchestration; the only place a decision is made	contain detection logic
<code>trace.py</code>	the event log	format anything for a human

Where to change things

you want to...	edit
add a tool	<code>sentinel/tool_model.yaml</code> (a tool absent from it gets an inferred spec)
change how strict a domain is	<code>PROFILES</code> in <code>sentinel/policy.py</code>
decide irreversible actions need no human	<code>confirm_irreversible</code> in the profile
add a detection signal	a function in <code>sentinel/signals.py</code> + a weight in <code>risk.py</code>
add a rewrite strategy	<code>sentinel/rewrite.py</code> , then add it to <code>candidates()</code>

Extension points

A different agent. Anything that produces `CandidateAction` objects works. The guard's interface is `begin_turn` / `observe` / `review` / `note_effect`; `api_adapter.py` drives it from the official v1 request, which is how Qwen3-8B reaches it.

A different environment. The guard needs only `(source_ref, text, trust, sensitivity)` per observation. Point `observe()` at a real tool bus and the defense works unchanged.

Learned weights. `risk.WEIGHTS` is a flat dict of log-odds priors. Fitting them needs a corpus larger than 49 scenarios — see TECHNICAL_REPORT § 4.6 for why we did not.